LOG AI -- Security Log Processing & Threat Detection Platform

Comprehensive Project Implementation & Presentation Report

> Document Purpose: Technical architecture walkthrough, active implementation record, and hackathon presentation (PPT) preparation guide based on the current codebase of LOG AI.

1. Project Overview (Executive Summary)

What Does LOG AI Do?

LOG AI is a high-performance, air-gapped security telemetry ingestion, normalization, and explainable threat detection platform. It collects raw system logs from heterogeneous network perimeter appliances--such as Cisco ASA firewalls, Fortinet FortiGate gateways, Suricata IDS/IPS sensors, and pfSense appliances--converts them into a standardized common schema (OCSF 1.1), detects suspicious activity using an in-memory Isolation Forest ML model, correlates related events into incident clusters, and presents plain-English explanations and step-by-step mitigation commands to security operators.

Core Problem Solved

1. **Log Format Tower of Babel**: Security teams deal with dozens of incompatible log formats (CEF, Syslog, CSV, KV-pairs, JSON), making automated threat correlation across vendor siloes nearly impossible.

2. **

machi

Primary Workflow

```
[ Edge Log Payload ]
```

2. Problem Statement

Enterprise Security Challenges Addressed

* **Heterogeneous Perimeter Appliance Ingestion**: Modern enterprise SOCs operate multi-vendor environments. Parsing firewall logs, IDS alerts, and network telemetry usually requires writing brittle, custom regex rules per device type.

* **

guara

3. Solution Overview & Data Flow

LOG AI implements an end-to-end 6-stage telemetry processing architecture:

```
+----------------------------------------------------------------------------------+
| 1. F
```

4. How the System Works (End-to-End Walkthrough)

1. **Ingestion Request (`app/routers/ingest.py`)**:

An H

5. Technical Architecture & Technology Stack

| Component | Technology | Purpose & Justification |

| :--- |

6. Core Features

1. Landing Page & Scroll-Driven Storytelling (`LandingPage.jsx`)

* **Purpose**: Introduces visitors to LOG AI through simple, clear language and Apple-style scroll-driven progressive storytelling.

* **In
as the

2. Unified SOC Dashboard (`DashboardPage.jsx`)

* **Purpose**: Gives security operators real-time visibility into overall threat posture, active incident count, pipeline latency, and live telemetry feeds.

* **In

3. Log Explorer (`LogExplorerPage.jsx`)

* **Purpose**: Search, filter, inspect, and paginate raw and normalized telemetry logs across multiple vendor parsers.

* **In

4. Forensic Investigation & Merkle Audit (`ForensicsPage.jsx`)

* **Purpose**: Verify SHA-256 cryptographic hashes of raw log payloads against the Merkle tree root hash to confirm zero data tampering.

* **In

5. Rule Studio (`RuleStudioPage.jsx`)

* **Purpose**: Allows analysts to configure custom detection rules, score multipliers, and pattern triggers.

* **In

6. AI Copilot & Explainable AI (XAI) (`CopilotWidget.jsx` & `app/xai/explainer.py`)

* **Purpose**: Provides plain-English explanations of flagged anomalies and step-by-step mitigation commands for active security incidents.

* **In

7. Financial Impact & ROI Estimator (`LandingPage.jsx`)

 * **Purpose**: Allows security leaders to calculate monthly financial savings ($) and reclaimed analyst hours based on daily log volume and device count.

 * **In

30) / 6

7. Supported Log Formats

LOG AI includes native parser support for top enterprise security perimeter appliances:

| Appliance / Format | Match Pattern / Signature | Extracted Fields | Standardized event_type |

| :--- |

8. OCSF 1.1 Normalization Schema

All parsed logs are mapped to the standard UnifiedEvent Pydantic model (app/normalization/schema.py):

```
class UnifiedEvent(BaseModel):
    ti
```

9. Detection & Anomaly Analysis (Isolation Forest ML)

Pure NumPy Isolation Forest Engine (`app/detection/engine.py`)

LOG AI features a custom, pure Python/NumPy implementation of the Isolation Forest anomaly detection algorithm.

1. **Feature Vector Extraction**:

For

2. **Isolation Tree Ensemble**:

Th
determ

3. **Feature Attribution Z-Scores**:

The
$Z \ge

---

10. Severity & Incident Correlation

Severity Threshold Mapping

* **CRITICAL** (≥ 90.0 Threat Score): Immediate threat requiring active mitigation.

* **H

Incident Clustering Heuristic (`app/detection/correlation.py`)

* **Grouping Key**: Scoped by `source_ip`.

* **R

---

11. Offline GeoIP Resolution (`app/services/geoip.py`)

LOG AI provides zero-network-latency offline IP geolocation:

* **P
Asia P

12. Security, Authentication & Multi-Tenancy

* **Authentication Endpoint (`app/routers/auth.py`)**: OAuth2 password bearer tokens using JWT (JSON Web Tokens) with 24-hour expiration.

* **P

13. Traditional SIEM vs. LOG AI Comparison

| Feature / Capability | Traditional Legacy SIEM | LOG AI Platform |

| :--- |

14. Presentation Slide Material (PPT Content)

This section provides a 12-slide presentation structure for hackathon demos:

Slide 1 -- Title Slide

* **Title**: LOG AI -- Security Log Processing & Threat Detection Platform

* **S

Slide 2 -- The Problem: Enterprise Log Chaos

* **Key Points**:

* Silo

Slide 3 -- The Solution: LOG AI Unified Architecture

* **Key Points**:

* Zer

Slide 4 -- Zero-Loss Ingestion & Forensic Integrity

* **Key Points**:

* Per

Slide 5 -- Multi-Vendor Parsing & OCSF 1.1 Schema

* **Key Points**:

* Nat

Slide 6 -- Pure NumPy Isolation Forest Anomaly Detection

* **Key Points**:

* 5-fe

Slide 7 -- 15-Minute Sliding Window Correlation

* **Key Points**:

* Gro

Slide 8 -- Explainable AI (XAI) & Actionable Playbooks

* **Key Points**:

* Z-s

Slide 9 -- Offline GeoIP & Threat Map

* **Key Points**:

* 100

Slide 10 -- Financial Impact & SOC ROI Estimator

* **Key Points**:

* Dua

Slide 11 -- Live System Demo & Technology Stack

* **Key Points**:

* Tec

Slide 12 -- Summary & Future Roadmap

* **Key Points**:

* Cor

15. Live Demo Script (3-Minute Sequence)

1. **0:00 - 0:45 | Landing Page & Interactive Vector Radar**:

* *Ac

16. Likely Judge Technical Questions & Answers

1. **Q: Why implement a custom Isolation Forest instead of using Scikit-Learn?**

* **A
enviro

17. Current Technical Limitations & Future Scope

Current Limitations

* **Correlation Scope**: Incident correlation is currently scoped per `source_ip`. It does not yet group attacks across distributed botnet subnets sharing different source IPs.

* **S

cluste

Future Scope

* **Automated Active Defense**: Adding direct API connectors to push firewall block rules directly to Cisco ASA and Fortinet routers upon incident creation.

* **
Redis/

18. Implementation File Map

```
log_ai/
```

```
??? ap
```

19. Final One-Page Presentation Summary

* **Problem**: Enterprise security teams face log format fragmentation across Cisco, Fortinet, Suricata, and pfSense devices, leading to alert fatigue and unexplainable anomaly alerts.

* **S
with p